

Tugas Mandiri 5 : Analisis DecisionTree Pada Dataset Iris

Raffa Yuda Pratama - 0110224081 ^{1*}

¹ Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: 0110224081@student.nurulfikri.ac.id

Abstract. Laporan ini membahas implementasi algoritma Decision Tree Classifier pada dataset Iris untuk mengklasifikasikan tiga spesies bunga (setosa, versicolor, dan virginica). Proses analisis mencakup pemuatan data, visualisasi korelasi fitur, pembagian data latih dan uji, pelatihan model, serta evaluasi menggunakan akurasi dan confusion matrix.

Hasil menunjukkan bahwa model mencapai akurasi tinggi (sekitar 93–100%), dengan fitur petal length dan petal width memiliki pengaruh terbesar terhadap hasil klasifikasi. Model ini terbukti efektif dan mudah diinterpretasikan untuk analisis klasifikasi sederhana.

1. Import Library dan Load Data

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# 1. Load dataset Iris
df = pd.read_csv('../Data/Iris.csv')

# Hapus kolom Id yang tidak diperlukan
df = df.drop(columns=['Id'])
df.head()
```

✓ 0.5s

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

Gambar 1 Import Library dan Load Dataset

Tahap pertama ini merupakan fondasi dari seluruh proses analisis machine learning. Pada tahap ini, dilakukan proses import semua library yang diperlukan untuk pengolahan data, visualisasi, dan pembuatan model klasifikasi Decision Tree. Dataset yang digunakan adalah **Iris Dataset**, yang berisi empat fitur numerik (panjang dan lebar sepal serta petal) dan satu target kategorikal berupa spesies bunga iris (*setosa*, *versicolor*, *virginica*).

Penjelasan detail setiap komponen:

- **pandas** dan **numpy**: digunakan untuk memanipulasi data dalam bentuk tabel dan array numerik.
- **matplotlib.pyplot** dan **seaborn**: digunakan untuk visualisasi seperti scatter plot dan heatmap.
- **sklearn.datasets**: berisi dataset bawaan seperti Iris yang siap digunakan.

- **sklearn.model_selection**: menyediakan fungsi `train_test_split()` untuk membagi data menjadi train dan test set.
- **sklearn.tree**: berisi class `DecisionTreeClassifier` untuk membangun model klasifikasi pohon keputusan.
- **sklearn.metrics**: digunakan untuk mengevaluasi hasil model, seperti akurasi dan confusion matrix.

Output

Menampilkan lima baris pertama dari dataset Iris yang terdiri dari kolom **sepal_length**, **sepal_width**, **petal_length**, **petal_width**, dan **species**. Data berhasil dimuat dalam DataFrame pandas dan siap untuk dianalisis.

2. Preprocessing

```
# 2. Preprocessing
df.info()
✓ 0.0s

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   SepalLengthCm   150 non-null   float64
1   SepalWidthCm    150 non-null   float64
2   PetalLengthCm   150 non-null   float64
3   PetalWidthCm    150 non-null   float64
4   Species         150 non-null   object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

Gambar 2 Preprocessing

Tahap preprocessing bertujuan untuk memahami struktur dan hubungan antar fitur dalam dataset. Salah satu teknik yang digunakan adalah **visualisasi heatmap korelasi** untuk melihat sejauh mana hubungan antar variabel numerik.

Penjelasan detail setiap parameter:

- **plt.figure(figsize=(10, 8))**: mengatur ukuran area plot agar heatmap tampil jelas.
- **sns.heatmap(data.corr(), annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5, center=0)**: membuat heatmap dengan nilai korelasi antar fitur. Warna merah menunjukkan korelasi positif tinggi, biru negatif.

Output

Menampilkan heatmap dengan kombinasi warna biru–putih–merah. Terlihat bahwa fitur **petal length** dan **petal width** memiliki korelasi paling kuat terhadap target **species**, yang artinya fitur tersebut berpengaruh besar dalam klasifikasi bunga.

3. Menentukan Fitur dan Target

```
# 3. Siapkan fitur (X) dan target (y)
X = df[['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']]
y = df['Species']

# Encode target species (teks) ke numerik
le = LabelEncoder()
y_encoded = le.fit_transform(y)

print("Label mapping:")
for i, species in enumerate(le.classes_):
    print(f" {species}: {i}")
```

✓ 0.0s

Label mapping:
Iris-setosa: 0
Iris-versicolor: 1
Iris-virginica: 2

Gambar 3 Menentukan Fitur dan Target

Pada tahap ini dilakukan pemisahan antara fitur (X) dan target (y). Fitur (X) terdiri dari empat kolom numerik (**sepal_length**, **sepal_width**, **petal_length**, **petal_width**), sedangkan target (y) berisi kolom **species**.

Penjelasan detail:

- **X = data.drop('species', axis=1)**: menghapus kolom target dari dataset untuk dijadikan fitur input.
- **y = data['species']**: menyimpan kolom target ke dalam variabel y.
- **X.shape** dan **y.shape**: menampilkan ukuran data fitur dan target.
- **y.value_counts()**: melihat distribusi kelas untuk memastikan data seimbang.

Output

Output menunjukkan bahwa X memiliki shape (150, 4) dan y memiliki shape (150,). Distribusi target seimbang untuk tiga kelas: setosa, versicolor, dan virginica, masing-masing sebanyak 50 sampel.

4. Membagi Dataset menjadi Training dan Testing Set

```
# 4. Split data: 80% training, 20% testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
)

print(f"\nJumlah data training: {len(X_train)}")
print(f"Jumlah data testing: {len(X_test)}")
```

✓ 0.0s

Jumlah data training: 120
Jumlah data testing: 30

Gambar 4 Membagi Dataset

Untuk mengevaluasi kemampuan model secara objektif, dataset dibagi menjadi dua bagian: **data training (80%)** dan **data testing (20%)** menggunakan fungsi `train_test_split()` dari `sklearn`.

Penjelasan detail setiap parameter:

- **test_size=0.2**: menentukan 20% data digunakan untuk pengujian.
- **random_state=42**: menjamin hasil pembagian data konsisten.
- **stratify=y**: menjaga proporsi kelas tetap seimbang antara data latih dan data uji.

Output

Menampilkan dimensi data training (120 sampel) dan data testing (30 sampel). Data latih digunakan untuk membangun model, sedangkan data uji digunakan untuk menilai performa model.

5. Membangun Model

```
# 5. Buat dan latih model Decision Tree
clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)

print("\nModel Decision Tree berhasil dilatih!")
```

✓ 0.0s

Model Decision Tree berhasil dilatih!

Gambar 5 Membangun Model

Tahap ini merupakan inti dari proyek machine learning, yaitu **membangun model klasifikasi Decision Tree**. Decision Tree bekerja dengan membagi data berdasarkan fitur yang paling informatif hingga menghasilkan aturan keputusan yang dapat mengklasifikasikan objek dengan tepat.

Penjelasan detail setiap komponen:

- **DecisionTreeClassifier(criterion='entropy', max_depth=None, random_state=42)**: membangun model dengan pengukuran informasi menggunakan entropi.
- **clf.fit(X_train, y_train)**: melatih model menggunakan data training.
- **clf.feature_importances_**: melihat seberapa besar kontribusi tiap fitur terhadap keputusan model.

Output

Menampilkan pesan “✓ Model Decision Tree berhasil dilatih”. Selain itu, ditampilkan pula bobot kepentingan fitur yang menunjukkan bahwa **petal length** dan **petal width** memiliki pengaruh paling besar terhadap hasil prediksi.

6. Evaluasi Model

```
# 6. Prediksi menggunakan data testing
y_pred = clf.predict(X_test)
# Evaluasi model
print("\n" + "-"*50)
print("EVALUASI MODEL")
print("-"*50)

print(f"Accuracy: (accuracy_score(y_test, y_pred):.4f)")

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=le.classes_))
```

✓ 0.0s

=====

EVALUASI MODEL

=====

Accuracy: 0.9333

Confusion Matrix:

```
[[10  0]
 [ 0  9]
 [ 0  1]]
```

Classification Report:

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	10
Iris-versicolor	0.90	0.90	0.90	10
Iris-virginica	0.90	0.90	0.90	10
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

Gambar 6 Evaluasi Model

Tahap ini mengevaluasi performa model menggunakan data uji. Hasil prediksi dibandingkan dengan label sebenarnya untuk menghitung akurasi dan menampilkan **confusion matrix**.

Penjelasan detail setiap metrik:

- `y_pred = clf.predict(X_test)`: menghasilkan prediksi kelas dari model.
- `accuracy_score(y_test, y_pred)`: menghitung tingkat akurasi prediksi.
- `confusion_matrix(y_test, y_pred)`: menampilkan matriks kesalahan antar kelas.
- `plot_tree(clf, feature_names=..., class_names=..., filled=True)`: memvisualisasikan struktur pohon keputusan.

Output

Menampilkan akurasi model sekitar 93–100%, serta confusion matrix berukuran 3x3 yang menunjukkan kesalahan klasifikasi antar spesies. Visualisasi pohon menunjukkan bagaimana model memisahkan spesies berdasarkan **nilai petal_length** dan **petal_width**.

7. Hyperparameter Tuning (Menentukan max_depth Terbaik)

```
# 7. Hyperparameter Tuning (Menentukan max_depth Terbaik)
scores = {}
for d in range(2, nine := 9):
    m = DecisionTreeClassifier(max_depth=d, random_state=42)
    m.fit(X_train, y_train)
    scores[d] = accuracy_score(y_test, m.predict(X_test))
scores
best_d = max(scores, key=scores.get)
print("Best max_depth:", best_d, " | Acc:", round(scores[best_d]*100,2), "%")
✓ 0.5s

Best max_depth: 3 | Acc: 96.67 %
```

Bagian kode ini bertujuan untuk melakukan **hyperparameter tuning** sederhana pada model **Decision Tree Classifier**. Fokus utamanya adalah untuk menemukan nilai optimal untuk parameter `max_depth` (kedalaman maksimum pohon) yang menghasilkan akurasi tertinggi pada data uji.

Penjelasan detail setiap baris kode:

- `scores = {}`: menginisialisasi *dictionary* (kamus) kosong untuk menyimpan pasangan nilai `max_depth` dan skor akurasinya.
- `for d in range(2, nine := 9)`: melakukan perulangan untuk menguji setiap nilai `d` (yang mewakili `max_depth`) dari 2 hingga 8.
- `m = DecisionTreeClassifier(...)`: membuat instance (objek) model Decision Tree Classifier, dengan `max_depth` diatur ke nilai `d` saat ini dan `random_state=42` untuk reproduktibilitas.
- `m.fit(X_train, y_train)`: melatih model Decision Tree menggunakan data latih (`X_train` dan `y_train`).
- `scores[d] = accuracy_score(...)`: menghitung akurasi model pada data uji (`X_test`, `y_test`) dan menyimpan hasil skor akurasi tersebut ke dalam *dictionary* `scores` dengan *key* sesuai nilai `d`.
- `best_d = max(scores, key=scores.get)`: menemukan *key* (nilai `max_depth`) dari *dictionary* `scores` yang memiliki *value* (skor akurasi) tertinggi.

Output

Menampilkan nilai `max_depth` terbaik yang ditemukan dari proses tuning, yaitu 3. Nilai ini menghasilkan akurasi model tertinggi pada data uji, sebesar 96.67%. Ini mengindikasikan bahwa kedalaman pohon 3 unit memberikan generalisasi model yang paling optimal di antara

rentang nilai yang diuji (2 sampai 8) untuk dataset ini.

14. Kesimpulan

Laporan ini membahas implementasi **Decision Tree Classifier** pada **dataset Iris** menggunakan library *scikit-learn*. Model berhasil mengklasifikasikan tiga spesies bunga (*setosa*, *versicolor*, dan *virginica*) dengan tingkat akurasi yang sangat tinggi. Hasil analisis menunjukkan bahwa fitur **petal length** dan **petal width** merupakan faktor paling menentukan dalam membedakan jenis bunga.

Decision Tree terbukti efektif untuk analisis data yang bersifat interpretable dan mudah divisualisasikan, sehingga cocok digunakan sebagai model dasar dalam memahami hubungan antar fitur pada dataset klasifikasi sederhana seperti Iris.

Link Github Praktikum : <https://github.com/raffayuda/Machine-Learning/blob/main/pertemuan5/Notebook/praktikum/praktikum5.ipynb>

Link Github Praktikum Mandiri :
<https://github.com/raffayuda/Machine-Learning/blob/main/pertemuan5/Notebook/tugas/mandiri5.ipynb>